Complete Neural Network Mathematics & Formulae Guide

A Full Reference Document From Initial Matrix Operations to Attention & Backpropagation

1. MATRIX OPERATIONS LIBRARY (MATRIX.CS)

1.1 Dense Matrix Multiplication (Multiply)

Computes row-column dot products to combine feature spaces between left matrix A and right weight matrix B.

$$Output_{i, j} = \sum_{k=0}^{K - 1} A_{i, k} \cdot B_{k, j}$$

Variable	Definition
$A_{i, k}$	Element at row i, column k in left input matrix A (dimension M × K).
$B_{k, j}$	Element at row k, column j in right weight matrix B (dimension K × N).
$Output_{i, j}$	Resulting element at row i, column j (dimension M × N).
K	Shared inner dimension bridging the matrices.

Why it's needed: Matrix multiplication is the foundational operator in deep learning that projects, rotates, and combines input features into different vector dimensions.

1.2 Matrix Transposition (Transpose)

Flips a matrix over its diagonal, turning rows into columns and columns into rows.

$$A^T_{j, i} = A_{i, j}$$

Variable	Definition
$A_{i, j}$	Original element at row i, column j in matrix A (dimension M × N).
$A^T_{j, i}$	Element at row j, column i in transposed matrix A^T (dimension N × M).

Why it's needed: Essential for matrix alignment during Attention score calculations ($Q \cdot K^T$) and gradient calculations during backpropagation.

1.3 Element-Wise Matrix Scaling (Scale)

Multiplies every individual cell in a matrix by a constant scalar value α.

$$Output_{i,j} = \alpha \cdot A_{i,j}$$

Variable	Definition
α	Scalar multiplier (e.g., Learning Rate η or temperature factor).
$A_{i,j}$	Individual matrix cell entry.

Why it's needed: Scales gradient updates to control weight adjustment step sizes during learning.

1.4 Matrix Addition & Residual Broadcasting (Add)

Adds two matrices element-by-element with support for 1-row bias vector broadcasting.

$$Output_{i,j} = A_{i,j} + B_{r,j} \quad \text{where} \quad r = \begin{cases} 0, & \text{if B has 1 row} \\ i, & \text{otherwise} \end{cases}$$

Variable	Definition
$A_{i,j}$	Primary feature matrix cell entry.
$B_{r,j}$	Added feature matrix or bias vector cell entry.

Why it's needed: Adds bias terms to linear transformations and calculates residual skip connections (X + Attention(X)).

2. TOKENIZER & EMBEDDING LAYER

2.1 Vocabulary Indexing (f_tokenize)

Translates raw string words into discrete integer addresses.

$$f_{tokenize}(w) = \begin{cases} \textbf{Vocab}[w], & \text{if } w \in \textbf{Vocab} \\ 1, & \text{if } w \notin \textbf{Vocab} \ (\langle UNK \rangle) \end{cases}$$

Variable	Definition
w	Input string word extracted from text.
$Vocab$	Dictionary mapping clean word strings to unique integer token IDs [0, V - 1].
1 ($\langle UNK \rangle$)	Fallback index reserved for unknown placeholder tokens.

Why it's needed: Bridges text strings to numerical integer addresses required for fetching continuous vectors.

2.2 Vector Embedding Lookup

Retrieves continuous vector representations for discrete token indices.

$$E(t_i) = W_{embed}[t_i, :] \in \&mathbb;R^{1 \times d_{model}}$$

Variable	Definition
t_i	Integer Token ID at position i in the input sequence.
W_{embed}	Embedding weight matrix of shape ($V \times d_{model}$).
d_{model}	Latent vector dimension size (e.g., 4 or 16).

Why it's needed: Converts discrete integer token IDs into dense geometric vectors where semantic meaning can be learned.

3. ACTIVATIONS & TRANSFORMER ARCHITECTURE

3.1 Sigmoid Activation Function

Squeezes real numbers into an S-curve bounded between 0 and 1.

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

Variable	Definition
x	Unrestricted raw scalar input.
e	Euler's mathematical constant (≈ 2.71828).

Why it's needed: Introduces non-linearity and produces probability-like gating values for attention score calculations.

3.2 Swish Activation Function

Smooth, non-monotonic gating activation function.

$$Swish(x) = x \cdot \sigma(x) = \frac{x}{1 + e^{-x}}$$

Variable	Definition
x	Input hidden state scalar.
$\sigma(x)$	Standard Sigmoid function applied to x.

Why it's needed: Allows small negative gradients to flow during backpropagation, avoiding dead neuron problems found in ReLU.

3.3 SwiGLU Gated Activation

Modern feed-forward layer combining Gate and Value neural branches.

$$SwiGLU(X) = (X \cdot W_{gate} + b_{gate}) \odot Swish(X \cdot W_{up} + b_{up})$$

Variable	Definition
X	Input hidden state matrix of shape ($M \times d_{model}$).
W_{gate}, W_{up}	Gate and Value projection weight matrices ($d_{model} \times d_{hidden}$).
$\odot$	Hadamard Product (element-wise matrix multiplication).

Why it's needed: Dynamically controls feature passage in feed-forward layers to increase model capacity.

3.4 Self-Attention Mechanism

Calculates pairwise contextual relationships between tokens in a sequence.

$$Q = X \cdot W_Q, \quad K = X \cdot W_K, \quad V = X \cdot W_V$$

$$Attention(Q, K, V) = \sigma(Q \cdot K^T) \cdot V$$

Variable	Definition
Q, K, V	Query ("searching for"), Key ("contains"), and Value ("content") projection matrices.
K^T	Transposed Key matrix ($d_{model} \times M$).
$\sigma(Q \cdot K^T)$	Pairwise affinity matrix measuring relationship strength between tokens.

Why it's needed: Enables tokens to dynamically pull contextual meaning from surrounding words in a sentence.

3.5 Residual (Skip) Connections

Additive bypass around neural processing layers.

$$Output_{res} = X + SubBlock(X)$$

Variable	Definition
X	Input state matrix before sub-layer processing.
$SubBlock(X)$	Output generated by Self-Attention or SwiGLU sub-block.

Why it's needed: Prevents vanishing gradients, allowing information and error signals to flow uninterrupted across deep networks.

4. PREDICTIONS, LOSS & BACKPROPAGATION

4.1 Softmax Probability Distribution

Converts raw logit prediction scores into probabilities that sum to 1.0.

$$P(y_{i,j}) = \frac{e^{z_{i,j} - max(z_i)}}{\sum_{k=0}^{V-1} e^{z_{i,k} - max(z_i)}}$$

Variable	Definition
$z_{i,j}$	Raw output logit score for token j at sequence index i.
$max(z_i)$	Maximum logit score in row i (subtracted to prevent exponential overflow).
$P(y_{i,j})$	Probability score assigned to word j (bounded in [0, 1]).

Why it's needed: Normalizes raw output logits into valid probability distributions over the vocabulary.

4.2 Cross-Entropy Loss Function

Quantifies error between predicted probabilities and target token.

$$Loss = -ln(P(y_{target}))$$

Variable	Definition
$P(y_{target})$	Model's predicted probability assigned to the correct target word.
ln	Natural logarithm function (base e).

Why it's needed: Computes a single penalty score representing prediction error. Higher confidence in the correct word results in lower loss.

4.3 Logit Gradient Derivation (LM Head Backward Pass)

Analytical derivative of Cross-Entropy combined with Softmax.

$$\frac{\partial Loss}{\partial z_j} = \begin{cases} P(y_{target}) - 1.0, & \text{if } j = target \\ P(y_j), & \text{if } j \neq target \end{cases}$$

Variable	Definition
$\partial Loss / \partial z_j$	Gradient rate of change of loss with respect to output logit z_j.
$- 1.0$	Derivative subtraction offset applied exclusively to the target token index.

Why it's needed: Calculates the starting error signal that is propagated backward through the network to update weights.

4.4 Backpropagation Weight Update Rule

Adjusts parameters using Stochastic Gradient Descent (SGD) to reduce loss over time.

$$W_{new} = W_{old} - \eta \cdot \frac{\partial Loss}{\partial W}$$

Variable	Definition
W_{old}	Current weight parameter matrix inside a neural layer.
η	Learning Rate factor (e.g., 0.05).
$\partial Loss / \partial W$	Calculated gradient matrix showing weight error sensitivity.
W_{new}	Updated weight parameter matrix.

Why it's needed: The fundamental learning step in deep learning. Modifies weights in the direction that lowers training error.